



RAMCO INSTITUTE OF TECHNOLOGY

Approved by AICTE, New Delhi & Affiliated to Anna University

NAAC Accredited with 'A+' Grade & An ISO 9001: 2015 Certified Institution

NBA Accredited UG Programs: CSE, EEE, ECE and MECH

Department of Computer Science and Engineering

Academic Year: 2025-2026(Odd Semester)

INNOVATIVE TEACHING PRACTICES

Degree, Semester & Branch: V Semester B.E. CSE

Course Code & Title: CCS375 Web Technologies

Name of the Faculty member: Dr. S. Poornam

Date & Time: 11.09.2025 & 10:30 A.M

Topic: Understanding Servlet Lifecycle

Course Outcome: CO3

Type of Activity: Roleplay Activity

Justification

This innovative classroom activity helps students visualize the Servlet life cycle particularly the execution of `init()`, `doGet()`, and `destroy()` methods through a Traffic Light Simulation. By linking servlet behavior to a real-world analogy (traffic signals), students gain a clear mental model of initialization, request handling, and termination phases of a servlet. The activity fosters active learning, conceptual visualization, and teamwork, bridging abstract server-side concepts with tangible, observable behavior.

Time Allotted for the Activity: 20 Minutes

Details of the Implementation

The classroom floor was divided into three colored zones to represent the servlet lifecycle phases:

- Green Light Zone → `init()` phase (Servlet Initialization)
- Yellow Light Zone → `doGet()` phase (Handling Multiple Requests)
- Red Light Zone → `destroy()` phase (Servlet Destruction)
- One student acted as the Server Container (Tomcat).
- Another student acted as the Servlet.
- Several students represented Clients sending HTTP requests.

Green Light Phase – Servlet Initialization (`init()`):

- When the green light card was raised, the “Server Container” activated the “Servlet.”
 - The servlet student announced, “Servlet initialized — ready to handle requests!”
 - The instructor explained that this occurs once per servlet loading when the container starts.
- ### **Yellow Light Phase – Request Handling (`doGet()`):**
- Each client student raised a small paper slip labeled “GET /HelloServlet” or “GET /DrumServlet.”
 - For every raised slip, the yellow light card was shown, and the servlet responded:

- “Request received and response sent!”
- The teacher emphasized that doGet() executes multiple times—once for each client request.
- This phase was repeated several times to reinforce the idea of frequent request handling.

Red Light Phase – Servlet Destruction (destroy()):

- Finally, when the server container raised a red light card, the servlet said:
- “Server shutting down... resources released!”
- The instructor clarified that the destroy() method runs only once when the servlet is unloaded or the server stops.
- Paper cards were used to represent HTTP requests and responses, making the data flow visual.
- A “Log Sheet” was maintained on the board to record the sequence of method calls — init() → doGet() → destroy().
- Students repeated the request phase to observe the difference in frequency between initialization, request handling, and destruction.

Learning Outcome:

- Students clearly understood the execution order, purpose, and repetition pattern of servlet lifecycle methods through active roleplay, visual aids, and repetition.
- Students clearly understood the **role of the servlet container** in managing lifecycle stages.

CO – PO / PSO Mapping

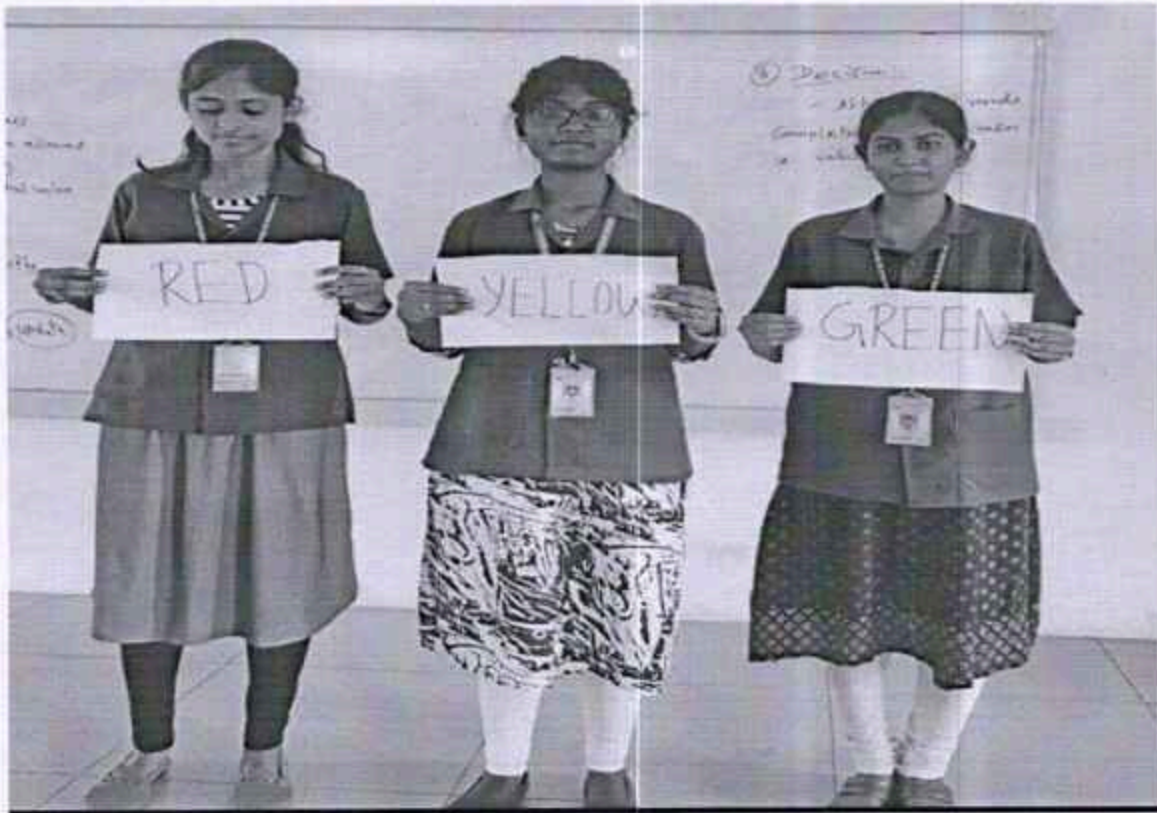
CO	PO1	PO2	PO3	PO4	PSO1
CO1	2	2	3	2	2

(1 – Low, 2 – Moderate, 3 – High)

Innovative practice	PO1	PO2	PO3	PO4	PSO1
	2	2	3	2	2
Justification for correlation	Students developed a conceptual understanding of servlet lifecycle and its relation to the servlet container.	Students analysed request-response cycles and identified which methods execute once and which execute repeatedly.	Students practiced designing and debugging servlet-based logic in alignment with lifecycle methods.	Students learned to associate real-time events (traffic lights) with server states.	Students applied lifecycle concepts in practical servlet coding sessions using Tomcat and NetBeans.

PO/PSO mapped: PO1, PO2, PO3, PO4, PSO1

Images / Screenshot of the practice:



Reflective Critique:

- ❖ Students reported that the servlet lifecycle roleplay provided a **clear and memorable understanding** of how the `init()`, `doGet()`, and `destroy()` methods operate within a web server environment.
- ❖ They appreciated the **visual and interactive representation** of the three lifecycle phases using colored zones, which helped them connect theoretical servlet concepts to real-world execution flow.
- ❖ Many students mentioned that **acting out** the roles of the Server Container, Servlet, and Clients clarified the **difference in frequency** between initialization, multiple request handlings, and destruction.
- ❖ Students also shared that the **use of light signals (green, yellow, red)** helped them easily recall the correct sequence and purpose of each lifecycle method even after the session.

Benefits of the Practice:

- ❖ This activity effectively enhanced students' understanding of the **Servlet lifecycle** by transforming an abstract server-side concept into a **tangible, experiential process**.
- ❖ The activity promoted **active learning** and encouraged students to visualize how a servlet behaves inside a container.

- ❖ By simulating multiple client requests, students developed a clearer grasp of how the **doGet() method executes repeatedly**, reinforcing the difference between one-time and recurring method calls.
- ❖ The use of visual aids such as colored lights, paper requests, and log sheets fostered **better retention and engagement**.
- ❖ The activity also encouraged **peer interaction and collaboration**, helping students articulate the servlet workflow confidently during later lab sessions.

Challenges Faced in Implementation:

- ❖ One key challenge was that **only a few students could participate actively** at a time (as servlet, server, and clients), while others remained observers.
- ❖ This limited overall participation, reducing the scope for all students to experience each role firsthand.
- ❖ Managing the **timing and coordination** of request repetitions also required careful facilitation to maintain the flow and attention.
- ❖ To improve future iterations, the activity could include **multiple simultaneous servlet groups or rotating roles**, allowing more students to engage directly and deepen their understanding collectively.

References

- <https://docs.oracle.com/javaee/7/api/javax/servlet/Servlet.html>
- <https://www.javatpoint.com/life-cycle-of-a-servlet>
- <https://www.geeksforgeeks.org/servlet-life-cycle/>
- <https://en.wikipedia.org/wiki/Role-playing>
- <https://www.youtube.com/watch?v=UAAPQ6wxxis>
- <https://serc.carleton.edu/introgeo/roleplaying/whatis.html>


Signature of Faculty Member


HOD